TASKUP: GERENCIADOR DE TAREFAS

Thiago Vieira Alves Caldera
Frederico Barbosa da Silva
Gabriel Adriano Pontes
Olicio Souza Santos Junior
Sabrina Rayssa Teixeira
Prof. Ricardo Bicalho

RESUMO

Este artigo apresenta o TaskUp, um gerenciador de tarefas pessoais formado por uma API RESTful e por uma interface que a consome. O objetivo do trabalho é praticar desenvolvimento backend, modelagem de banco de dados e criação de APIs REST em um sistema pequeno. A API foi feita em Python com Django e Django REST Framework e usa o SQLite como banco de dados. O controle de versão é feito com Git e GitHub. A API utilizam HTML, CSS, JavaScript e Flutter. Com a API é possível cadastrar, listar, consultar, editar e excluir tarefas.

Palavras-chave: API REST; Python; Django; Django REST Framework; SQLite; HTML; CSS; JavaScript; Flutter; Git; GitHub; gerenciamento de tarefas.

ABSTRACT

This paper presents TaskUp, a personal task manager made of a RESTful API and an interface that uses it. The goal of the work is to practice backend development, database modeling and REST API design in a small system. The API was built with Python, Django, and Django REST Framework, and uses SQLite as its database. Version control is managed using Git and GitHub. The web interface and mobile application are built using HTML, CSS, JavaScript, and Flutter. It allows users to create, list, view, edit and delete tasks. So far, the backend and the database are ready and were tested manually with HTTP requests. The interface and the cloud deployment are left for the next stages.

Keywords: REST API;Django; Django REST; SQLite;task management.

1. INTRODUÇÃO

Um gerenciador de tarefas é uma aplicação fácil de entender e boa para praticar os conceitos da disciplina. O cliente, que é a interface, fica separado do servidor, que guarda os dados e aplica as regras, e os dois conversam por meio de uma API. Assim, a mesma API pode atender uma interface web ou um aplicativo móvel.

O TaskUp foi desenvolvido na unidade curricular Sistemas Distribuídos e Mobile. O objetivo é criar uma aplicação para organizar tarefas e, com ela, aplicar conceitos de backend, banco de dados e APIs RESTful. Como metas, o grupo definiu implementar as operações básicas sobre as tarefas e, depois, integrar o backend a uma interface.

2. FUNDAMENTAÇÃO TEÓRICA E TECNOLOGIAS

2.1. API RESTful

REST é um estilo de arquitetura apresentado por Fielding (2000). Nele, cada coisa que a aplicação manipula é um recurso com um endereço (URL), e as operações usam os métodos do HTTP: GET para consultar, POST para criar, PUT para alterar e DELETE para excluir. A resposta traz um código de status, como 200 (sucesso), 201 (criado), 400 (requisição inválida) e 404 (não encontrado). No TaskUp, o recurso é a tarefa, acessada em /tarefas.

2.2. Django, Django REST, SQLite.

Foram usados a linguagem Python e o framework Django, juntamente com o Django REST Framework, que simplifica a criação de rotas e a construção de APIs RESTful. O banco de dados é o SQLite, que guarda tudo em um único arquivo e não exige instalar um servidor de banco. O desenvolvimento do frontend web utiliza HTML, CSS e JavaScript,Todo o versionamento de código e a colaboração do projeto são realizados através do Git e hospedados no GitHub.

3. MODELAGEM E IMPLEMENTAÇÃO

3.1. Funcionalidades

As funcionalidades do TaskUp estão na Tabela 1 e são atendidas pelos endpoints da Tabela 2.

Tabela 1 - Funcionalidades do TaskUp.

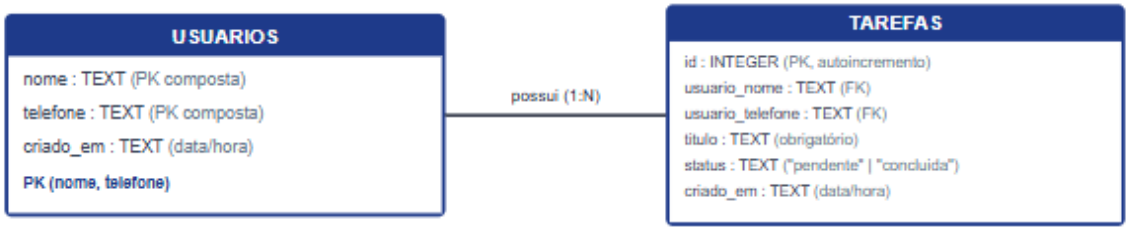
Código	Funcionalidade	Descrição
RF01	Cadastrar tarefa	Cria uma tarefa com título (obrigatório) e descrição (opcional), com status pendente.
RF02	Listar tarefas	Retorna todas as tarefas, das mais recentes para as mais antigas.
RF03	Visualizar detalhes	Retorna os dados de uma tarefa específica.
RF04	Editar tarefa	Altera o título, a descrição ou o status da tarefa.
RF05	Marcar como concluída	Muda o status da tarefa para concluída.
RF06	Excluir tarefa	Remove a tarefa do banco de dados.

Fonte: Elaborado pelos autores (2026).

3.2. Modelo de dados

O banco possui duas tabelas relacionadas. usuarios armazena nome, telefone e data de criação, com chave primária composta. tarefas armazena título, descrição opcional, status e data de criação, além de referenciar o usuário por nome e telefone. A Figura 1 apresenta o esquema.

Figura 1 - Diagrama entidade-relacionamento do TaskUp.



Fonte: Elaborado pelos autores (2026).

3.3. API do backend

Os modelos e migrações do Django definem as tabelas usuarios e tarefas; as views e serializers do Django REST definem as rotas da API. Usuários são identificados por nome e telefone, e cada tarefa pertence a um usuário. O servidor valida os dados recebidos, incluindo título obrigatório e status permitido.

Tabela 2 - Endpoints principais da API.

Método	Rota	Função	Respostas
GET	/usuarios	Lista usuários cadastrados	200
POST	/usuarios	Cadastra novo usuário	201, 409
GET	/tarefas	Lista tarefas	200, 400
GET	/tarefas/:id	Detalha tarefa específica	200, 404
POST	/tarefas	Cadastra nova tarefa	201, 400, 404
PUT	/tarefas/:id	Edita tarefa e status	200, 400, 404
DELETE	/tarefas/:id	Exclui tarefa do sistema	204, 404

Fonte: Elaborado pelos autores (2026).

4. RESULTADOS PARCIAIS

4.1. Teste do backend

O backend desenvolvido com Django REST foi executado localmente e testado com requisições HTTP feitas manualmente. O cadastro de uma tarefa retornou 201, com status pendente, e o cadastro sem título retornou 400. A mudança do status para concluída funcionou, e um status inválido retornou 400. A listagem trouxe as tarefas cadastradas. A exclusão retornou 204 e uma nova consulta à mesma tarefa retornou 404.

4.2. Próximas etapas

Faltam o frontend web (HTML, CSS, JavaScript) e o (Flutter) que consomem a API, a publicação do backend e do frontend em um serviço de nuvem e os testes funcionais com cliente e servidor funcionando juntos.

5. CONSIDERAÇÕES FINAIS

Nesta etapa, a API do TaskUp desenvolvida em Python e Django REST permite cadastrar e consultar usuários e executar operações CRUD em tarefas. O banco SQLite valida os dados e vincula cada tarefa a um usuário. O versionamento de código é feito com Git e GitHub. As próximas etapas são concluir a integração com os frontends web e publicar o sistema em nuvem. Trabalhos futuros incluem login, categorias, prazos e notificações.

6. REFERÊNCIAS

- DJANGO. **Django documentation**. Disponível em: https://docs.djangoproject.com/. Acesso em: 12 out. 2026.
- DJANGO REST FRAMEWORK. **Django REST framework documentation**. Disponível em: https://www.django-rest-framework.org/. Acesso em: 12 out. 2026.
- FIELDING, Roy Thomas. **Architectural styles and the design of network-based software architectures**. 2000. Tese (Doutorado) - University of California, Irvine, 2000.
- FLUTTER. **Flutter documentation**. Disponível em: https://docs.flutter.dev/. Acesso em: 12 out. 2026.
- PYTHON. **Python documentation**. Disponível em: https://docs.python.org/3/. Acesso em: 12 out. 2026.
- SQLITE. **SQLite documentation**. Disponível em: https://www.sqlite.org/docs.html. Acesso em: 12 out. 2026.